



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Reporte Técnico

Modelado y Programación



- Guzmán Arias Alexis Uriel
- Oviedo Juárez Mitsué
- Sánchez Fragoso Sofía

1. Definición del Problema

El programa recibe una imagen `.bmp` con un fondo de color sólido y una colección arbitraria de figuras geométricas de colores sólidos, cuyos colores son distintos entre sí y entre el fondo. Las figuras no tienen degradados, ni suavizado en los bordes, ni se traslapan entre ellas.

El programa debe procesar la imagen e identificar cada figura presente, clasificándola en una de cuatro categorías:

- **C**: Cuadriláteros (cuadrados, rectángulos, rombos, trapezoides).
- **T**: Triángulos (equiláteros, isósceles, escalenos, rectángulos).
- **O**: Círculos.
- **X**: Otros (Cualquier figura que no caiga en las categorías anteriores).

El programa debe reportar al usuario para cada figura detectada su clasificación y su color correspondiente en formato hexadecimal (`#RRGGBB`).

2. Arsenal

Decidimos hacer nuestro proyecto en **Python** debido a su facilidad para implementar y por las bibliotecas de manejo de datos y visión por computadora. Aunque es un lenguaje interpretado, las operaciones matemáticas se asignan a las bibliotecas compiladas en C/C++, dándonos un tiempo de ejecución eficiente.

Las bibliotecas que utilizamos y su justificación técnica son las siguientes:

- **OpenCV (cv2)**: Se utilizó para la detección de contornos (`cv2.findContours`), análisis de polígonos (`cv2.approxPolyDP`), procesamiento de las imágenes a color y el renderizado de los resultados.
- **NumPy (numpy)**: Nos permite manipular las imágenes como arreglos de N dimensiones. Se utilizó para realizar operaciones rápidas sobre los arreglos, detección del color de fondo por frecuencias y cálculo de diferencias absolutas por cada canal de color.
- **Pillow (PIL)**: Utilizada para convertir los arreglos de NumPy/OpenCV en imágenes compatibles con la interfaz.
- **Tkinter (tkinter)**: Es la biblioteca estándar de Python para manejar interfaces gráficas. Se usó para la selección interactiva de archivos (`filedialog`) y el reporte de resultados con formato usando objetos como `scrolledtext`.

3. Análisis del problema

Para desarrollar una solución nos basamos en las especificaciones y supuestos definidos sobre las imágenes de entrada, identificando los casos límite a considerar y definiendo los requisitos funcionales y no funcionales del programa.

3.1. Supuestos sobre las imágenes de prueba

- **Fondo uniforme:** La imagen contiene un fondo de un único color sólido.
- **Colores sólidos e independientes:** Cada figura posee un color sólido, diferente tanto del fondo como de las demás figuras.
- **Bordes definidos:** No existe anti-aliasing ni gradientes de color intermedios en los bordes de las figuras.
- **Sin traslape:** Las figuras no se traslapan entre sí.
- **Variación arbitraria:** El tamaño, ángulo de rotación y posición de cada figura en la imagen son totalmente arbitrarios.

3.2. Casos límite identificados

- **Fondo de color arbitrario:** No se asume que el fondo siempre sea un mismo color para todas las pruebas y no hay indicación previa de que color tiene el fondo.
- **Variación de color:** Una figura puede tener un color que contraste muy poco con el fondo haciendo que una máscara global del fondo pueda identificarla incorrectamente como parte del fondo.
- **Figuras adyacentes (tocándose en los bordes):** Cuando dos figuras de distinto color son adyacentes, una máscara global del fondo las detectaría como un único contorno.
- **Figuras con agujeros o geometrías complejas:** Una figura puede contener agujeros o geometrías complejas las cuales deben ser correctamente clasificadas.
- **Clasificación de círculos:** Un círculo dibujado en píxeles puede generar un número variable de vértices al aproximar su polígono, de igual forma una figura distinta podría tener una cantidad de vértices comparable con los de un círculo, por lo que contar vértices no garantiza la clasificación de círculos.
- **Imagen sin figuras:** Una imagen de prueba puede no contener figuras y solamente ser un fondo de color sólido.
- **Imagen con demasiadas figuras:** Una imagen de prueba puede contener una gran cantidad de figuras.
- **Rutas no válidas o archivos dañados:** La ruta seleccionada por el usuario puede ser nula, un tipo de archivo inválido o dirigir a un archivo dañado.

3.3. Requisitos funcionales

Los requisitos funcionales describen las acciones que realiza el programa para procesar las entradas y las salidas que debe entregar de acuerdo a nuestra solución.

1. **Entrada de imágenes:** El programa debe permitir la carga de imágenes en formato `.bmp` para su procesamiento.
2. **Procesamiento y detección de figuras:** El programa debe identificar y aislar las figuras presentes en la imagen reconociendo el color del fondo de manera automática.
3. **Clasificación de figuras:** El programa debe clasificar cada figura detectada en una de cuatro categorías:
 - **C:** Cuadriláteros (cuadrados, rectángulos, rombos, trapezoides).
 - **T:** Triángulos (equiláteros, isósceles, escalenos, rectángulos).
 - **O:** Círculos.
 - **X:** Otros (Cualquier figura que no caiga en las categorías anteriores).
4. **Identificación de color:** El programa debe obtener el color sólido correspondiente de cada figura y expresarlo en formato hexadecimal (`#RRGGBB`).
5. **Renderizado e imagen de salida:** El programa debe entregar como salida la imagen procesada marcando visualmente los bordes de cada figura detectada.
6. **Reporte de resultados:** El programa debe entregar como salida un reporte que indique el total de figuras encontradas, detallando para cada una su tipo (C, T, O, X), posición (x, y) y color en formato hexadecimal.

3.4. Requisitos no funcionales

Los requisitos no funcionales definen los atributos de calidad, rendimiento y restricciones que garantizan que nuestra solución sea confiable, utilizable y mantenible.

1. **Velocidad de procesamiento y eficiencia:** El análisis completo de una imagen de tamaño razonable (lectura, procesamiento, clasificación y renderizado) debe ejecutarse de forma casi instantánea (menos de 1 segundo), aprovechando las operaciones de NumPy y las llamadas de OpenCV.
2. **Amigabilidad y claridad en la experiencia de usuario:** La interacción debe ser intuitiva mediante una interfaz gráfica en Tkinter. Se evita obligar al usuario a escribir rutas manualmente en la terminal gracias al uso del explorador de archivos (`filedialog`), mostrando previsualizaciones de las imágenes y la opción de descargar la imagen con los bordes resaltados y de copiar el reporte al portapapeles.
3. **Tolerancia a fallas y manejo de entradas inválidas:** Ante rutas mal escritas, archivos no soportados, imágenes corruptas o la cancelación en la selección de archivos, el programa no debe finalizar con errores o detenerse. Debe capturar las excepciones y notificar al usuario de manera clara.
4. **Claridad del código y mantenibilidad:** La solución debe implementar una estructura limpia dividida en módulos con responsabilidades separadas (preprocesamiento, detección, clasificación e interfaz gráfica). Esto permite agregar nuevas categorías de figuras geométricas o ajustar criterios de procesamiento de forma individual sin modificar la lógica general.
5. **Portabilidad y facilidad de ejecución:** La aplicación debe ser capaz de ejecutarse sin modificaciones en cualquier entorno o sistema operativo (Linux, Windows, macOS) que tenga una instalación estándar de Python 3 y las bibliotecas especificadas en las dependencias.

4. Selección de la mejor alternativa

Para la detección, separación y clasificación de las figuras en las imágenes de prueba, intentamos dos soluciones distintas basadas en la biblioteca OpenCV y NumPy.

4.1. Solución inicial: Detección por espacios de color y luminancia (Descartada)

En nuestra primera solución implementamos un enfoque basado en la conversión de la imagen a escala de grises y la detección de bordes mediante el operador de Canny. Sin embargo, en imágenes con fondos de colores intensos (Ej. rojo), la luminancia del fondo resultaba ser muy cercana a la de ciertas figuras de colores distintos, provocando que las figuras se perdieran en la imagen binaria como si fueran parte del fondo.

Para corregir esto, se implementó un segundo intento utilizando el espacio de color LAB y gradientes de Sobel:

1. Se determinó el color del fondo identificando el color más frecuente en la matriz completa de la imagen.
2. Se calculó la diferencia máxima canal por canal BGR para obtener una primera máscara binaria con un umbral fijo .
3. Se convirtió la imagen al espacio de color LAB y se aplicó el operador de Sobel (3×3) para cada canal (L , A y B), encontrando la magnitud del gradiente y seleccionando la máxima entre los tres canales.
4. La matriz resultante de bordes se dilató con un núcleo de 3×3 y se restó de la máscara binaria principal para forzar la separación de figuras adyacentes.

Razones del descarte: Aunque esta solución mejoró la separación de objetos, dependía de umbrales arbitrarios de magnitud y de las operaciones morfológicas agresivas. Esto provocaba que figuras pequeñas o geometrías complejas sufrieran deformaciones en sus bordes, afectando la precisión del conteo de vértices mediante `approxPolyDP` y generando clasificaciones erróneas.

4.2. Alternativa elegida: Separación directa por BGR y Discontinuidad de bordes

Aprovechando los supuestos del problema (fondo sólido único y colores planos sin anti-aliasing ni gradientes), diseñamos un algoritmo más limpio, rápido y exacto:

1. **Detección del color de fondo por perímetro:** En lugar de analizar la imagen completa, escogemos únicamente los píxeles de los cuatro bordes de la imagen usando `np.unique`. Como el fondo en general rodea toda la imagen, elegir el color más frecuente de este subconjunto determina consistentemente el color del fondo sin interferencia por la existencia de figuras grandes cerca del centro.
2. **Separación binaria exacta:** Se separa el fondo utilizando `cv.inRange` sobre el valor *BGR* exacto del fondo obtenido y se invierte la máscara con `cv.bitwise_not`. Esto elimina la dependencia de umbrales de luminancia o diferencias progresivas.
3. **Separación de figuras adyacentes por diferencia de color:** Para resolver el caso límite en que dos figuras de distinto color están en contacto, se calculan las diferencias absolutas de color entre píxeles adyacentes de forma horizontal (Δx) y vertical (Δy) con `cv.absdiff`:

$$\text{Diferencia}_H = |\text{Imagen}[i, j] - \text{Imagen}[i + 1, j]|, \quad \text{Diferencia}_V = |\text{Imagen}[i, j] - \text{Imagen}[i, j + 1]|$$

Cualquier cambio de color mayor a cero entre píxeles vecinos marca la frontera exacta entre dos objetos y se resta de la máscara de objetos (`cv.subtract`), logrando una separación física exacta sin deformar la geometría de las figuras.

4. **Manejo de jerarquías y agujeros en figuras:** Se utiliza `cv.findContours` con el modo `RETR_CCOMP`, el cual organiza los contornos en una jerarquía de dos niveles. Esto permite identificar si un contorno es externo (figura) o interno (perforación o agujero).
5. **Criterios de clasificación geométrica:** Cada contorno recibe una clasificación de figura en base a los siguientes criterios:
 - **Círculos (O):** Se evalúa la proporción entre el área del contorno y el área del círculo mínimo envolvente (`cv.minEnclosingCircle`). Si esta proporción es $\geq 95\%$, se clasifica como círculo, evitando los errores de conteo de vértices en curvas dibujadas por píxeles.
 - **Triángulos (T) y Cuadriláteros (C):** Se aplica el algoritmo de Ramer-Douglas-Peucker (`cv.approxPolyDP`) con una precisión del 1 % del perímetro. Si el polígono tiene 3 vértices se asigna a **T**, y si tiene 4 vértices a **C**.
 - **Otros (X):** Cualquier polígono que no sea un círculo, con número de vértices diferente a 3 o 4, o que tenga agujeros internos, se asigna a la clase **X**.

4.3. Justificación de la elección

Elegimos esta solución por las siguientes ventajas sobre la solución inicial:

- **Confiabilidad y precisión:** Al no usar suavizados, dilataciones morfológicas ni filtros de gradiente (Sobel/Canny), los contornos encontrados coinciden exactamente píxel por píxel con las figuras de la imagen original.
- **Eficiencia en tiempo:** Elimina la conversión a espacios de color como LAB y las varias iteraciones de filtros, procesa todas las figuras al mismo tiempo evitando generar máscaras individuales por cada figura en la imagen, reduciendo el tiempo de procesamiento a unos pocos milisegundos por imagen.
- **Evita parámetros configurados a mano:** Funciona de forma analítica para cualquier combinación de colores en el fondo o en las figuras sin requerir el ajuste de umbrales fijos como los de la solución inicial.

5. Diagrama de flujo del algoritmo

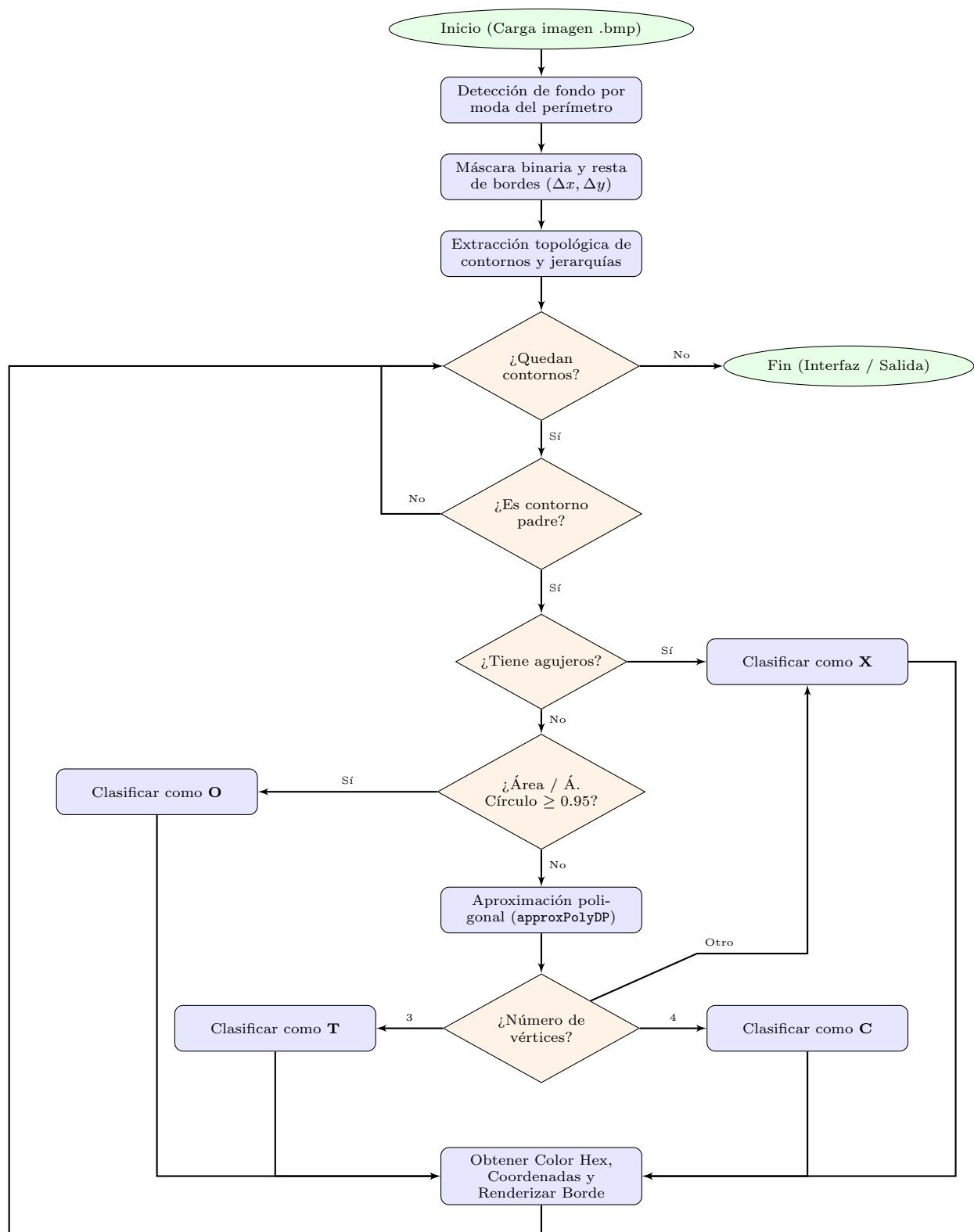


Figura 1: Diagrama de flujo del algoritmo